CODE WALKTHROUGH · SILT AI DATA ANALYST

# Login and team security: `backend/app/security.py`

79 lines that decide **who** is calling the API and **which team's data** they may see. Every protected endpoint in the backend goes through this file.

SIZE
**79 lines**, 6 functions

LIBRARIES
**bcrypt**, **PyJWT**, FastAPI, SQLAlchemy

USED BY
**103** user checks, **85** team checks

TOKEN
**JWT HS256**, valid 7 days

## 1. At a glance

| # | Function | Lines | Type | One-line job |
|---|---|---|---|---|
| 1 | `hash_password` | 18-19 | helper | Plain password → bcrypt hash (sign-up) |
| 2 | `verify_password` | 22-23 | helper | Does the typed password match the hash? (login) |
| 3 | `create_access_token` | 26-34 | helper | Issue a signed JWT after sign-up / login |
| 4 | `decode_access_token` | 37-41 | helper | Verify a JWT's signature and expiry |
| 5 | `get_current_user` | 44-55 | dependency | Who is calling? (authentication) |
| 6 | `get_current_team` | 58-79 | dependency | Which team, and are they a member? (authorization) |

Functions 1-3 are used by the sign-up and login endpoints. Functions 5-6 protect everything else.

### Imports (lines 4-15)

| Import | Used for |
|---|---|
| `bcrypt` | Password hashing and checking |

| Import | Used for |
| --- | --- |
| `jwt` (PyJWT) | Creating and verifying login tokens |
| `Depends, Header, HTTPException` | FastAPI: chain the checks, read request headers, return error codes |
| `Session`, `get_db` | Database session for loading users, teams and memberships |
| `config` | Secret key, algorithm and expiry (section 4) |
| `Team, TeamMember, User` | The database tables that are checked |

backend/app/security.py : lines 1–15 (docstring and imports)

```
"""Password hashing, JWT session tokens, and the FastAPI auth dependencies
that every team-scoped endpoint depends on.
"""
from __future__ import annotations

import datetime as dt

import bcrypt
import jwt
from fastapi import Depends, Header, HTTPException
from sqlalchemy.orm import Session

from app import config
from app.database import get_db
from app.models import Team, TeamMember, User
```

## 2. Functions, one by one

Each function uses the same format: Purpose, Signature, Inputs, Returns, Errors, How it works, Used by, and the real code.

### 1. `hash_password` helper

lines 18-19

| | |
|---|---|
| **PURPOSE** | Turns a plain password into a one-way bcrypt hash so the real password is never stored. |
| **SIGNATURE** | `hash_password(password: str) -> str` |
| **INPUTS** | `password` : the plain text the user typed at sign-up. |
| **RETURNS** | A 60-character string such as `$2b$12$...` containing the algorithm, cost, random salt and hash. |
| **ERRORS** | None of its own. bcrypt raises `ValueError` if the password is longer than 72 bytes (see section 6). |
| **HOW IT WORKS** | 1. Encode the password to UTF-8 bytes (bcrypt works on bytes).<br>2. `bcrypt.gensalt()` creates a new random salt, so two users with the same password get different hashes.<br>3. `bcrypt.hashpw()` computes the hash (deliberately slow, to resist guessing).<br>4. Decode the bytes back to a string so it can be saved in `users.password_hash` . |
| **USED BY** | `routers/auth.py` → `signup()` |
| **WHY** | Hashing is one-way: even someone who reads the database cannot get the passwords back. |

backend/app/security.py : lines 18–19

```
18  def hash_password(password: str) -> str:
19      return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")
```

### 2. `verify_password` helper

lines 22-23

| | |
|---|---|
| **PURPOSE** | Checks whether a typed password matches the stored hash. |
| **SIGNATURE** | `verify_password(password: str, password_hash: str) -> bool` |

| | |
|---|---|
| **INPUTS** | `password` : what the user typed at login. `password_hash` : the value stored at sign-up. |
| **RETURNS** | `True` if they match, otherwise `False` . |
| **ERRORS** | None of its own. bcrypt raises `ValueError` for a password over 72 bytes. |
| **HOW IT WORKS** | 1. Encode both values to bytes.<br>2. `bcrypt.checkpw()` reads the salt from inside the stored hash, re-hashes the typed password with it, and compares the two in constant time. |
| **USED BY** | `routers/auth.py` → `login()` |
| **WHY** | There is no way to "decrypt" a hash, so checking means re-hashing and comparing. |

backend/app/security.py : lines 22–23

```
22  def verify_password(password: str, password_hash: str) -> bool:
23      return bcrypt.checkpw(password.encode("utf-8"), password_hash.encode("utf-8"))
```

## 3. `create_access_token` helper

lines 26-34

| | |
|---|---|
| **PURPOSE** | Creates the signed JWT login token returned after a successful sign-up or login. |
| **SIGNATURE** | `create_access_token(user_id: int, email: str) -> str` |
| **INPUTS** | `user_id` and `email` of the user who just logged in. |
| **RETURNS** | A JWT string with three parts, `header.payload.signature` . |
| **ERRORS** | None. |
| **HOW IT WORKS** | 1. Take the current UTC time.<br>2. Build the payload: `sub` (the user id, as a string), `email` , `iat` (issued at) and `exp` (expires at = now + `JWT_EXPIRE_MINUTES` , 7 days by default).<br>3. Sign it with `JWT_SECRET_KEY` using HS256 and return it. |
| **USED BY** | `routers/auth.py` → `signup()` and `login()` |
| **WHY** | The server stores no session: the signature proves the token was issued by this server and not changed. |

backend/app/security.py : lines 26–34

```
26  def create_access_token(user_id: int, email: str) -> str:
27      now = dt.datetime.utcnow()
28      payload = {
```

```
        "sub": str(user_id),
        "email": email,
        "iat": now,
        "exp": now + dt.timedelta(minutes=config.JWT_EXPIRE_MINUTES),
    }
    return jwt.encode(payload, config.JWT_SECRET_KEY, algorithm=config.JWT_ALGORITHM)
```

## 4. decode_access_token `helper`

lines 37-41

| | |
|---|---|
| **PURPOSE** | Checks a token's signature and expiry and returns its payload. |
| **SIGNATURE** | `decode_access_token(token: str) -> dict` |
| **INPUTS** | `token` : the JWT sent by the client. |
| **RETURNS** | The payload dict, e.g. `{"sub": "7", "email": ..., "iat": ..., "exp": ...}` . |
| **ERRORS** | `401 Invalid or expired token` if the signature is wrong, the token was edited, it has expired, or it is not a JWT. |
| **HOW IT WORKS** | 1. `jwt.decode()` verifies the signature with the secret key and checks `exp` .<br>2. Only HS256 is accepted ( `algorithms=[...]` ), so a forged token using another algorithm is rejected.<br>3. Any `PyJWTError` becomes an HTTP 401. |
| **USED BY** | `get_current_user()` |

backend/app/security.py : lines 37–41

```
def decode_access_token(token: str) -> dict:
    try:
        return jwt.decode(token, config.JWT_SECRET_KEY, algorithms=[config.JWT_ALGORITHM])
    except jwt.PyJWTError as e:
        raise HTTPException(401, "Invalid or expired token") from e
```

## 5. get_current_user `FastAPI dependency`

lines 44-55

| | |
|---|---|
| **PURPOSE** | Works out **who** is making the request. Any endpoint that needs a logged-in user declares it with `Depends(get_current_user)` . |
| **SIGNATURE** | `get_current_user(authorization: str \| None = Header(...), db: Session = Depends(get_db)) -> User` |
| **INPUTS** | `authorization` : the `Authorization: Bearer <token>` header, read by FastAPI.<br>`db` : a database session from `get_db` . |

| | |
|---|---|
| **RETURNS** | The `User` row from the database. |
| **ERRORS** | `401 Missing bearer token` (no header, or it does not start with "Bearer "); `401 Invalid or expired token` (from `decode_access_token`); `401 User no longer exists` (account deleted after the token was issued). |
| **HOW IT WORKS** | 1. Check the header exists and starts with `bearer` (case-insensitive).<br>2. Split once on the space to get the token.<br>3. Decode and verify it.<br>4. Load the user by the id in `sub`.<br>5. Return the user, or 401 if they no longer exist. |
| **USED BY** | 103 endpoint parameters across the backend, e.g. `/api/auth/me`, `/api/me/workspaces`, and every team endpoint through `get_current_team`. |
| **WHY** | Checking that the user still exists means a deleted account cannot keep using an old token. |

backend/app/security.py : lines 44–55

```
44  def get_current_user(
45      authorization: str | None = Header(default=None),
46      db: Session = Depends(get_db),
47  ) -> User:
48      if not authorization or not authorization.lower().startswith("bearer "):
49          raise HTTPException(401, "Missing bearer token")
50      token = authorization.split(" ", 1)[1].strip()
51      payload = decode_access_token(token)
52      user = db.get(User, int(payload["sub"]))
53      if user is None:
54          raise HTTPException(401, "User no longer exists")
55      return user
```

## 6. `get_current_team` FastAPI dependency

lines 58-79

| | |
|---|---|
| **PURPOSE** | Works out **which team** the request is for and proves the user belongs to it. This is the core of multi-tenant isolation: one team can never read another team's data. |
| **SIGNATURE** | `get_current_team(x_team_id: int \| None = Header(..., alias="X-Team-Id"), user: User = Depends(get_current_user), db: Session = Depends(get_db)) -> Team` |
| **INPUTS** | `x_team_id`: the `X-Team-Id` header (FastAPI converts it to an int). `user`: from `get_current_user`, so authentication always runs first. |
| **RETURNS** | The `Team` row. Endpoints then filter every query by `team.id`. |

| | |
|---|---|
| **ERRORS** | `400 X-Team-Id header is required`; `404 Team not found`; `403 Not a member of this team` (including invited-but-not-accepted, since status must be `active`). Plus all the 401s from `get_current_user`. |
| **HOW IT WORKS** | 1. If the header is missing, return 400.<br>2. Load the team; 404 if it does not exist.<br>3. Look for a `team_members` row with this team, this user and `status == "active"`.<br>4. If there is none, 403. Otherwise return the team. |
| **USED BY** | 85 endpoint parameters, e.g. dataset upload and listing, asking a question, dashboards, audit trail. |
| **WHY** | The client chooses the team id, so it cannot be trusted; this membership query is what stops a user from simply typing another team's id. |

backend/app/security.py : lines 58–79

```python
def get_current_team(
    x_team_id: int | None = Header(default=None, alias="X-Team-Id"),
    user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
) -> Team:
    if x_team_id is None:
        raise HTTPException(400, "X-Team-Id header is required")
    team = db.get(Team, x_team_id)
    if team is None:
        raise HTTPException(404, "Team not found")
    member = (
        db.query(TeamMember)
        .filter(
            TeamMember.team_id == team.id,
            TeamMember.user_id == user.id,
            TeamMember.status == "active",
        )
        .first()
    )
    if member is None:
        raise HTTPException(403, "Not a member of this team")
    return team
```

## 3. One request, end to end

1. **Client sends**
   `GET /api/datasets` with `Authorization: Bearer eyJ...` and `X-Team-Id: 3`.

2. **FastAPI resolves dependencies**
   The endpoint asks for `team = Depends(get_current_team)`, which itself needs `get_current_user`, which needs `get_db`.

3. **get_current_user**
   Reads the token → `decode_access_token` checks the signature and expiry → loads user 7.

4. **get_current_team**
   Reads `X-Team-Id: 3` → loads team 3 → checks user 7 is an **active** member.

5. **Endpoint runs**
   Receives `user` and `team` already checked, and filters with `Dataset.team_id == team.id`.

6. **Any check fails**
   An `HTTPException` is raised and the endpoint body never runs. The client gets 400 / 401 / 403 / 404 with a JSON `detail`.

How an endpoint uses it (simplified from `list_datasets` in `main.py`):

```
@app.get("/api/datasets")
def list_datasets(db = Depends(get_db),
                  user: User = Depends(get_current_user),
                  team: Team = Depends(get_current_team)):
    ... .filter(Dataset.team_id == team.id) ...
```

## 4. Error codes this file produces

| Code | Message | Raised in | When |
|---|---|---|---|
| **400** | `X-Team-Id header is required` | `get_current_team` | User has no active team selected (the bug we fixed on the Datasets page). |
| **401** | `Missing bearer token` | `get_current_user` | Not logged in, or header not in `Bearer <token>` form. |

| Code | Message | Raised in | When |
|---|---|---|---|
| **401** | `Invalid or expired token` | `decode_access_token` | Token tampered with, signed with another key, or older than 7 days. |
| **401** | `User no longer exists` | `get_current_user` | Account deleted after the token was issued. |
| **403** | `Not a member of this team` | `get_current_team` | Real team, but the user is not an active member. |
| **404** | `Team not found` | `get_current_team` | No team with that id. |

## 5. Configuration and tests

| Setting (config.py) | Value | Meaning |
|---|---|---|
| `JWT_SECRET_KEY` | env var; random per process if unset (with a warning) | Signs and verifies tokens. Must be set in production, otherwise every restart logs everyone out. |
| `JWT_ALGORITHM` | "HS256" | HMAC with SHA-256: one shared secret both signs and verifies. |
| `JWT_EXPIRE_MINUTES` | 10080 (7 days) | How long a token stays valid. |

backend/app/config.py : lines 81–94 (JWT settings)

```
81  JWT_SECRET_KEY = os.getenv("JWT_SECRET_KEY", "")
82  if not JWT_SECRET_KEY:
83      # Ephemeral per-process secret: fine for local dev (tokens just don't
84      # survive a restart), but every deployment should set a real one --
85      # otherwise a restart invalidates every session, and multiple workers
86      # wouldn't agree on tokens at all.
87      JWT_SECRET_KEY = secrets.token_hex(32)
88      warnings.warn(
89          "JWT_SECRET_KEY is not set -- using a random one for this process only. "
90          "Set JWT_SECRET_KEY in the environment for any real deployment.",
91          stacklevel=2,
92      )
93  JWT_ALGORITHM = "HS256"
94  JWT_EXPIRE_MINUTES = int(os.getenv("JWT_EXPIRE_MINUTES", str(60 * 24 * 7)))  # 7 days
```

| Test file | What it proves |
|---|---|
| `tests/test_auth.py` | sign-up then login then `/me`; duplicate email; short password; wrong password; unknown email; `/me` needs a token |

| Test file | What it proves |
| --- | --- |
| `tests/test_team_isolation.py` | datasets scoped to own team; another team's dataset is 404; foreign team header is 403; no token is 401 even with a team header; invite rules |

## 6. Known limitations

**Passwords over 72 bytes**

bcrypt 5.0 (the pinned version) raises `ValueError` for passwords longer than 72 bytes. I confirmed this for both `hashpw` and `checkpw`. Sign-up only checks the minimum length (8), and there is no global error handler, so such a password would cause an HTTP 500 instead of a clear 400 message. Fix: reject passwords over 72 bytes in `signup()`.

**No token revocation**

Tokens cannot be cancelled before they expire (see the logout question).

`datetime.utcnow()`

Deprecated in Python 3.12 in favour of `datetime.now(timezone.utc)`. It still works today.

## 7. Likely viva questions

**Why bcrypt and not SHA-256 for passwords?**

SHA-256 is designed to be fast, so an attacker can try billions of guesses a second. bcrypt is deliberately slow and adds a random salt to every hash, which makes guessing and pre-computed tables impractical.

**What is inside our JWT, and is it encrypted?**

`sub` (user id), `email`, `iat` and `exp`. It is **signed, not encrypted**: anyone can read the payload, but nobody can change it without the secret key. That is why no password or secret is put in it.

**Why is `sub` stored as a string?**

The JWT standard says `sub` is a string, and PyJWT validates that. The code converts it back with `int(payload["sub"])`.

### What is `Depends()`?

FastAPI's dependency injection. An endpoint lists what it needs; FastAPI runs those functions first and passes in the results. If one raises an exception, the endpoint never runs. Because `get_current_team` depends on `get_current_user`, you cannot get a team without logging in first.

### A user changes X-Team-Id to another team's id. What happens?

`get_current_team` finds no active membership row for that user and team and returns 403. On top of that, endpoints check that each row they load belongs to `team.id` and return 404 otherwise.

### How does logout work if the server keeps no sessions?

The client deletes its token. The token itself stays valid until it expires (7 days). Revoking tokens early would need a deny-list, which this project does not have yet.

### Why 401 vs 403?

401 means "we don't know who you are" (no or bad token). 403 means "we know who you are, but you are not allowed" (not a member of this team).

Generated from the repository source on 16 October 2026. Every code block is copied from the file and line range in its caption.